CS5730 Operating Systems: 2020 Midterm Solutions

2020 · CS5730 Operating Systems (2020/12/1; the paper is titled "Final")

Only the photographed pages are available (Q1–Q8).

1. Short Answer: Your answers should be as concise as possible; aim for three or four sentences.

Q1.1 (5 pt) Ordinarily the `exec()` system call follows the `fork()`. Explain what would happen if a programmer were to inadvertently place the call to `exec()` before the call to `fork()`. 2020

ANSWER

exec() **replaces the calling process's entire memory image** (code, data, heap, stack) with the program named in its parameters. On success, **it never returns**.

So if exec() is placed before fork(), the new program starts running in place of the original one, under the **same process ID**. The rest of the original code, **including the fork() call, is gone and never executed**. **No child process is created**, and the original program's remaining work is not done.

Only if exec() **fails** (returns −1) does execution continue to the fork().

Q1.2 (5 pt) What happens if one thread of a multi-threaded process call `fork()`? What happens if one thread of a multi-threaded process call `exec()`? 2020

ANSWER

fork(): this depends on the system. Some UNIX systems provide **two versions** of fork(). One **duplicates all threads** of the process. The other (the POSIX behaviour) **duplicates only the thread that called fork()**, so the child is single-threaded. If exec() is called right after fork(), duplicating only the calling thread is enough, because exec replaces everything anyway. If the child does not call exec, duplicating all threads may be needed.

exec(): exec works as usual. The program named in its parameter **replaces the entire process, including all of its threads**. The other threads are destroyed, and the new program starts with a single thread.

Q1.3 (5 pt) What is processor affinity? Explain why it needed to be considered during process scheduling on multiple CPU systems. 2020

ANSWER

Processor affinity: a process has an affinity for the processor it is currently running on. It is kept on (or restricted to) that same CPU rather than migrated to another one.

Why it matters on multiple-CPU systems: while a process runs, its recently used data fills that processor's **cache**. If the scheduler migrates it to another CPU, the old cache contents must be **invalidated** and the new processor's cache must be **repopulated**. That causes many cache misses and memory stalls, which is expensive. Keeping the process on the same CPU reuses the "warm" cache. On NUMA systems it also keeps the process close to its local memory.

Soft affinity: the OS tries to keep the process on the same CPU but may migrate it. **Hard affinity:** the process specifies the set of CPUs it may run on. The scheduler must balance affinity against **load balancing** (push/pull migration), which moves processes from busy to idle CPUs.

Q2 (10 pt) One technique for implementing lottery scheduling works by assigning processes lottery tickets, which are used for allocating CPU time. Whenever a scheduling decision has to be made, a lottery ticket is chosen at random, and the process holding that ticket gets the CPU. The BTV operating system implements lottery scheduling by holding a lottery 50 times each second, with each lottery winner getting 20 milliseconds of CPU time (20 milliseconds × 50 = 1 second). Describe how the BTV scheduler can ensure that higher-priority threads receive more attention from the CPU than lower-priority threads. 2020

ANSWER

Give **higher-priority threads more lottery tickets**. At each of the 50 drawings per second, the winning ticket is chosen uniformly at random. A thread holding *k* of the *T* tickets therefore wins with probability k/T, and on average receives about **k/T of the CPU time** (k/T × 50 × 20 ms per second).

Example: high-priority thread H has 30 tickets and low-priority L has 10. H wins about 75% of the drawings (≈ 750 ms per second) and L about 25% (≈ 250 ms). The scheduler can also adjust priority dynamically by giving or taking away tickets, e.g. extra tickets to a thread that has waited long. Every thread with at least one ticket eventually wins, so there is no starvation.

Q3 (10 pt) A variation of the round-robin scheduler is the **regressive round-robin scheduler**. This scheduler assigns each process a time quantum and a priority. The initial value of a time quantum is 50 milliseconds. However, every time a process has been allocated the CPU and uses its entire time quantum (does not block for I/O), 10 milliseconds is added to its time quantum, and its priority level is boosted. (The time quantum for a process can be increased to a maximum of 100 milliseconds.) When a process blocks before using its entire time quantum, its time quantum is reduced by 5 milliseconds, but its priority remains the same. What type of process, namely CPU-bound (such as batch processes) or I/O-bound (such as interactive processes), does the regressive round-robin scheduler favor? Explain. 2020

ANSWER

It favors CPU-bound processes.

- A process that **uses its entire quantum** (does not block for I/O) is CPU-bound behaviour. It is **rewarded**: +10 ms of quantum, up to 100 ms, **and** a priority boost.

- A process that **blocks before using its quantum** is I/O-bound / interactive behaviour. It is **penalized**: −5 ms of quantum, and its priority is **not** raised.

So over time CPU-bound (batch) processes get longer time slices and higher priority, and I/O-bound processes get shorter slices and stay at lower priority. This is the opposite of the usual goal of favoring interactive processes.

Q4 (10 pt) Consider the following code segment: 2020

```
pid_t pid;

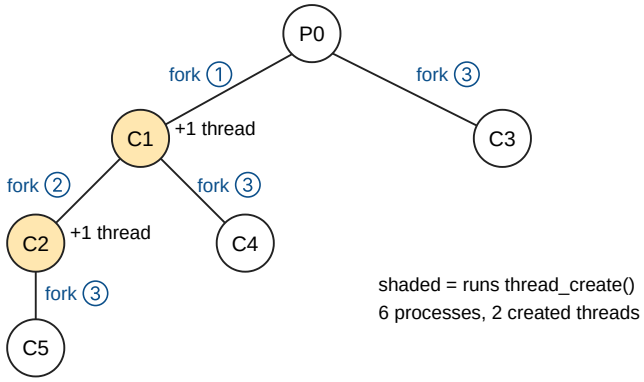
pid = fork();
if (pid == 0) { /* child process */
    fork();
    thread_create( . . .);
}
fork();
```

a) How many unique processes are created?
b) How many unique threads are created?

ANSWER

Trace each fork (the original process is P0):

- fork ① gives P0 and child C1. Only C1 has pid == 0.
- Inside the if, C1 calls fork ②, creating C2. **C1 and C2 each call thread_create()**, so 2 threads are created.
- The final fork ③ is executed by P0, C1 and C2, creating C3, C4 and C5.



(a) 6 unique processes (P0, C1–C5; 5 newly created if the original is not counted).
(b) 2 unique threads created by thread_create(), one in C1 and one in C2. The last fork() duplicates only the calling thread (POSIX), so it creates no extra threads.

Q5 (15 pt) Discuss how the following pairs of scheduling criteria conflict in certain settings. 2020

Q5.1 CPU utilization and response time 2020

ANSWER

CPU utilization vs. response time. CPU utilization is highest when context switches are rare, as with FCFS or a long quantum, because each switch is pure overhead. Good response time for interactive users needs **frequent preemption** (a short RR quantum) so every user gets the CPU quickly. That causes many context switches, so the CPU spends more time switching and less doing useful work, which lowers utilization.

Q5.2 Average turnaround time and maximum waiting time 2020

ANSWER

Average turnaround time vs. maximum waiting time. Average turnaround is minimized by running the **shortest jobs first** (SJF/SRTF). But a long job can be postponed again and again as short jobs keep arriving, so it may **starve**. Its waiting time, the **maximum** waiting time, becomes very large (unbounded). Bounding the maximum wait (e.g. with FCFS or aging) raises the average turnaround.

Q5.3 I/O device utilization and CPU utilization 2020

ANSWER

I/O device utilization vs. CPU utilization. Keeping devices busy means running **I/O-bound** processes as soon as their I/O completes, so they can issue the next request quickly. This means frequently preempting CPU-bound processes, and the context-switch overhead lowers CPU utilization. Maximizing CPU utilization means letting CPU-bound jobs run long without switching, which leaves the I/O devices **idle** while I/O-bound jobs wait for the CPU.

Q5.4 Deadline and fairness 2020

ANSWER

Deadline vs. fairness. To meet deadlines, the scheduler must always give the CPU to the most urgent process (earliest deadline or real-time priority), regardless of how much CPU it has already had. Other processes may wait a long time or **starve**, which is unfair. A

fair scheduler gives everyone an equal share, so urgent tasks may not get enough CPU in time and **miss their deadlines**.

Q5.5 Throughput and context switch overhead 2020

ANSWER

Throughput vs. context-switch overhead. Throughput (jobs completed per unit time) is improved by finishing short jobs quickly, which needs preemption: SJF/SRTF or a small RR quantum. Every preemption adds **context-switch overhead**, during which no useful work is done. Minimizing switches (FCFS, large quantum) traps short jobs behind long ones (**convoy effect**), so fewer jobs complete per unit time. If the quantum is too small, overhead itself eats the throughput.

Q6 (10 pt) A student is asked to write a program which will create a hundred POSIX threads. Each thread will print out the value of the parameter passed from `main()` which are 1, 2, 3, up to 100 respectively. Consider the following code fragment he wrote, please verify its correctness. If incorrect, please correct the error. 2020

```
void *doSomething(void *y) {
    ...
    printf("%d\n", *y);
    ...
}

int main() {
    pthread_t t1[100];          /* declare a thread */
    int param1, i;

    param1 = 1;
    for (i = 0; i < 100; i++) {
        param1++;
        pthread_create(&(t1[i]), NULL, doSomething, &param1);
    }
    ...
}
```

ANSWER

Incorrect. Problems:

1. **All 100 threads receive the same address** `&param1`. main keeps incrementing param1 while the threads run, so each thread prints whatever value param1 holds when that thread happens to run. The output has duplicates and missing numbers (often many 100s), not 1...100.
2. main never calls **pthread_join**, so it may return (ending the process and all its threads) before the threads print.
3. `*y` dereferences a `void *`, so it needs a cast to `int *`.

Corrected: give every thread its own argument and join all threads.

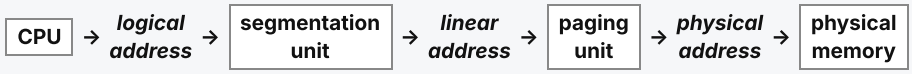
```
void *doSomething(void *y) {
    ...
    printf("%d\n", *(int *)y);
    ...
    pthread_exit(0);
}

int main() {
    pthread_t t1[100];            /* declare threads */
    int param[100], i;      /* one argument per thread */

    for (i = 0; i < 100; i++) {
        param[i] = i + 1;          /* 1, 2, ..., 100 */
        pthread_create(&(t1[i]), NULL, doSomething, &param[i]);
    }
    for (i = 0; i < 100; i++)
        pthread_join(t1[i], NULL);  /* wait for all */
    return 0;
}
```

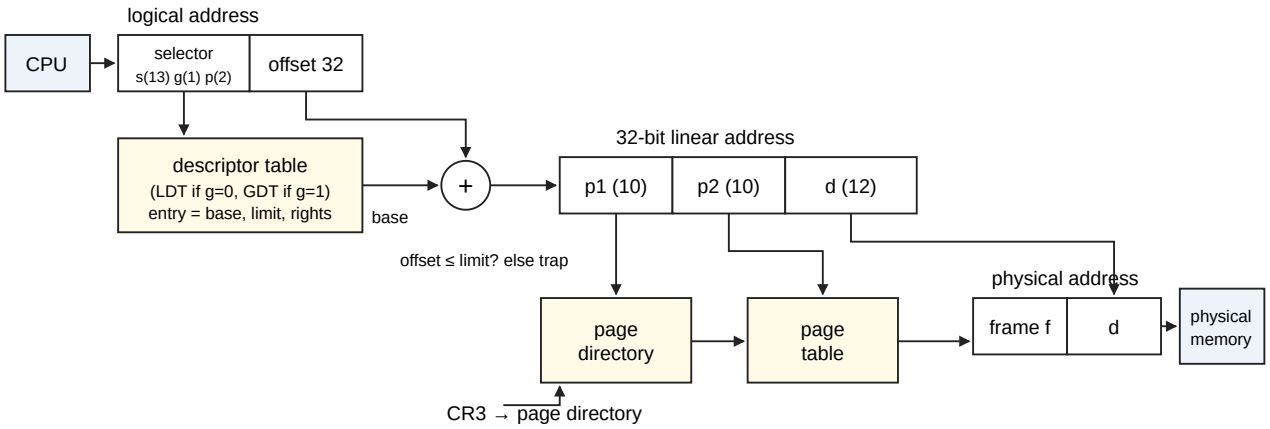
(If the numbers must also be printed *in order*, the threads must additionally be synchronized, e.g. by joining each thread right after creating it.)

7. Consider the Intel address translation scheme shown in the following.



Q7.1 (10 pt) Illustrate all the steps that the Intel 80386 takes in translating a logical address into a physical address. 2020

ANSWER



1. The CPU generates a **logical address = (selector, offset)**. The 16-bit selector has **s** = 13-bit segment number, **g** = 1 bit choosing GDT or LDT, and **p** = 2 protection bits.
2. The **segmentation unit** uses s and g to fetch the 8-byte **segment descriptor** from the **LDT** (private segments) or **GDT** (shared segments). Descriptors are cached in segment registers. It checks the offset against the segment **limit** and the access rights, trapping on a violation, then computes the **linear address = base + offset** (32 bits).

3. The **paging unit** splits the linear address into **p1 (10 bits) | p2 (10 bits) | d (12 bits)**. Register **CR3** points to the current **page directory**. p1 selects a directory entry, which points to a **page table**. p2 selects the page-table entry, which gives **frame f**.
4. **Physical address = f concatenated with d**, which is sent to memory. (For 4 MB pages the directory entry points straight at the frame, with a 22-bit offset. The TLB caches recent translations.)

Q7.2 (5 pt) What are the advantages to the operating system of hardware that provides such complicated memory translation hardware? 2020

ANSWER

- **Segmentation** matches the programmer's view (code, data and stack as separate logical units). It gives **per-segment protection** (e.g. read-only, execute-only code, privilege levels) and easy **sharing** of whole segments between processes.
- **Paging** on top removes **external fragmentation**. Segments need not be contiguous in physical memory; they are allocated in fixed-size frames, so allocation is simple.
- The OS gets **flexibility**: pages can be swapped or loaded on demand (virtual memory), shared at page level, and protected with page bits. The hardware does the work at memory speed, so the OS does not translate in software.

Q7.3 (5 pt) Are there any disadvantages to this address-translation system? If so, what are they? If not, why is it not used by every manufacturer? 2020

ANSWER

Yes.

- **Slower access:** without TLB hits, each reference needs a segment-descriptor lookup plus **two page-table memory accesses** (directory and table) before the data access itself.
- **Complexity and cost:** more hardware (segment registers, descriptor tables, MMU, TLB), more chip area and power. The OS must manage both segment tables and page tables.
- **Memory overhead** for the tables, and paging still has internal fragmentation.

Because of this cost and complexity, many manufacturers (most RISC designs, ARM, and x86-64 in practice with a flat segment model) use **paging only**. It delivers most of the benefit with simpler, faster hardware.

Q8 (10 pt) Assume we have a demand-paged memory. The page table is held in registers. It takes 8 milliseconds to service a page fault if an empty page is available or the replaced page is not modified, and 20 milliseconds if the replaced page is modified. Memory access time is 100 nanoseconds. Assume that the page to be replaced is modified 70 percent of the time. What is the maximum acceptable page-fault rate for an effective access time of no more than 200 nanoseconds? 2020

ANSWER

Let p = page-fault rate. Memory access = 100 ns. A page-fault service averages:

0.3 × 8 ms + 0.7 × 20 ms = 2.4 + 14 = **16.4 ms = 16,400,000 ns**

Effective access time: EAT = (1 − p) × 100 + p × 16,400,000 ≤ 200

100 + 16,399,900 p ≤ 200 → p ≤ 100 / 16,399,900

$p \leq 6.1 \times 10^{-6}$ (about 1 fault per 164,000 memory accesses).

Keep units consistent (ms → ns). Since the page table is in registers, a non-faulting access costs just one memory access (100 ns).